TP Projet 1 : Finance Agent et MDM Tiers et Structures

HESTIM | Data Governance | Projet "Enterprise Multi-Agent AI Governance Platform"
Membre : Junior (Finance) | Dataset : SEC Financial Statement Data | Prérequis : TP0 terminé

Objectifs pédagogiques

À l'issue de ce TP, vous serez capables de :

- Inspecter un dataset inconnu avant de construire quoi que ce soit
- Charger un dataset brut dans PostgreSQL avec Python
- Documenter un dataset avec les 4 types de métadonnées (fiche catalogue)
- Rédiger un dictionnaire de données selon les règles du cours
- Cataloguer et classifier un dataset dans OpenMetadata (owner, domaine, tags, glossaire)
- Appliquer les 5 tests du MDM pour choisir les entités maîtres
- Construire un Golden Record `company` avec une règle de priorité explicite
- Mesurer la qualité du référentiel (unicité, complétude, validité)

Votre mission

Vous êtes le **Data Steward du domaine Finance**. Le Finance Agent (partie 2) répondra à des questions comme "*Quel est le chiffre d'affaires 2024 de cette entreprise ?*". Pour qu'il puisse le faire **proprement**, il faut :

1. que les données SEC soient chargées et **documentées** ;
2. que chaque entreprise existe **une seule fois**, sous une forme fiable (le **Golden Record**) ;
3. que les postes financiers (chiffre d'affaires, actifs...) soient **standardisés**.

Votre MDM s'appelle **Tiers et Structures** :

- **Tiers** : les entreprises (Company / Issuer), qui sont des *personnes morales* (domaine **Parties**).
- **Structures** : le plan de comptes XBRL (domaine **Finance**) et les référentiels Currency, Country, SIC (domaine **Reference Data**).

Élément	Valeur
Dataset	`sec_financial_statements`
Classification	Public
Sensibilité	Low
PII	Non (ce sont des entreprises, pas des personnes)
Data Owner	Direction Finance
Data Steward	Vous
Relecteur de vos PR	Ivan (Investment)

Concepts clés pour ce TP

> **DÉFINITION : XBRL**
> Un format standard dans lequel les entreprises cotées aux États-Unis déposent leurs états financiers à la SEC. Chaque montant porte une **étiquette** (`tag`), par exemple `Revenues` (chiffre d'affaires) ou `Assets` (total des actifs). Ces tags forment un **plan de comptes** : c'est une donnée de **structure**, donc candidate au MDM.

> **DÉFINITION : CIK**
> Central Index Key. L'identifiant **unique** que la SEC donne à chaque entreprise (ex. Apple = `0000320193`). C'est la **clé métier** de votre Golden Record.

> **DÉFINITION : Règle de priorité (ou règle de survie)**
> Quand deux sources (ou deux dépôts) donnent deux valeurs différentes pour le même attribut, la règle dit **laquelle gagne**. Exemple : "*pour le nom de l'entreprise, le dépôt le plus récent gagne*". Sans règle écrite, chacun choisit au hasard et le référentiel n'est plus fiable.

ÉTAPE 1 : Découvrir le dataset

1.1 Télécharger

Le dataset officiel est publié par la SEC : **Financial Statement Data Sets**.

1. Ouvrir https://www.sec.gov/data-research/sec-markets-data/financial-statement-data-sets
2. Télécharger **un seul trimestre** récent (ex. `2025q1.zip`). Un trimestre suffit largement.
3. Dézipper dans `data/raw/sec/`. Vous obtenez 4 fichiers texte séparés par des tabulations :

Fichier	Contenu	Une ligne =
`sub.txt`	Les dépôts (submissions)	un rapport déposé par une entreprise
`num.txt`	Les montants	un chiffre (ex. Revenues 2024 = 391 Md$)
`tag.txt`	Le plan de comptes	une étiquette XBRL et son libellé
`pre.txt`	La présentation	où le chiffre apparaît dans le rapport (on ne l'utilise pas)

> **ATTENTION**
> `num.txt` fait plusieurs millions de lignes. Ne l'ouvrez pas dans Excel. On va le lire avec Python et **filtrer**. Le dossier `data/raw/` est ignoré par Git : les données ne vont jamais sur GitHub.

1.2 Inspecter les vraies colonnes

Créez `scripts/explore_sec.py` :

```
# Affiche les colonnes et un aperçu de chaque fichier SEC
import pandas as pd
```

```
for f in ["sub", "num", "tag"]:
    df = pd.read_csv(f"data/raw/sec/{f}.txt", sep="\t", dtype=str,
                     nrows=5000, quoting=3, encoding_errors="replace")
    print(f"\n===== {f}.txt : {len(df.columns)} colonnes")
    print(list(df.columns))
    print(df.head(3).T)
```

```
python scripts/explore_sec.py
```

Questions à noter dans votre compte rendu :

1. Quelle colonne relie `num.txt` à `sub.txt` ? (indice : elle commence par `a`)
2. Dans `sub.txt`, quelles colonnes décrivent l'entreprise (identifiant, nom, pays, secteur) ?
3. Dans `num.txt`, quelle colonne donne l'unité (USD, shares...) ? Quelle colonne donne la date ?
4. Dans `tag.txt`, que signifie `custom = 1` ? (indice : un tag inventé par l'entreprise)

> **EXPLICATION : pourquoi inspecter avant de construire ?**
> Le schéma des tables dépend des **vraies** colonnes. Les noms changent parfois d'une version à l'autre (par exemple la colonne `segments` n'existe que dans les fichiers récents). Construire sans regarder, c'est construire sur des suppositions.

> **CAPTURE D'ÉCRAN REQUISE**
> La sortie du script avec la liste des colonnes des 3 fichiers.

ÉTAPE 2 : Charger les données brutes dans PostgreSQL

On charge une **copie brute** dans le schéma `data`, avec le suffixe `_raw`. Brute veut dire : mêmes colonnes, tout en texte, aucune correction. Les corrections viendront dans le MDM (étape 5).

On **filtre seulement les lignes** pour garder une taille raisonnable :

- uniquement les rapports annuels `10-K` ;
- uniquement une dizaine de tags standards (`us-gaap`) ;
- uniquement les montants en `USD`, sans ventilation par filiale ou segment.

> **EXPLICATION : filtrer, c'est aussi de la gouvernance**
> Ce filtre change le contenu du dataset. Il doit donc être **documenté** dans les métadonnées opérationnelles (étape 3). Quelqu'un qui lit vos chiffres doit savoir qu'ils viennent uniquement des 10-K.

Installez les dépendances Python (`sqlalchemy` est déjà dans `requirements.txt`) :

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

Créez `scripts/load_sec.py` :

```
# Charge les fichiers SEC bruts dans PostgreSQL (schema data, tables *_raw)
import os
import pandas as pd
from dotenv import load_dotenv
from sqlalchemy import create_engine

load_dotenv()
engine = create_engine(
    f"postgresql+psycopg2://{os.getenv('POSTGRES_USER')}:{os.getenv('POSTGRES_PASSWORD')}"
    f"@localhost:{os.getenv('POSTGRES_PORT')}/{os.getenv('POSTGRES_DB')}"
)
RAW = "data/raw/sec"
OPTIONS = dict(sep="\t", dtype=str, quoting=3, encoding_errors="replace")

# Les postes financiers que l'on garde (notre plan de comptes simplifie)
TAGS = [
    "Revenues", "RevenueFromContractWithCustomerExcludingAssessedTax",
    "NetIncomeLoss", "OperatingIncomeLoss",
    "Assets", "Liabilities", "StockholdersEquity",
    "CashAndCashEquivalentsAtCarryingValue",
]

# 1. Depots : uniquement les rapports annuels
sub = pd.read_csv(f"{RAW}/sub.txt", **OPTIONS)
sub = sub[sub["form"] == "10-K"]
sub.to_sql("sec_sub_raw", engine, schema="data", if_exists="replace", index=False)
print("sec_sub_raw :", len(sub))

# 2. Plan de comptes : uniquement nos tags standards
tag = pd.read_csv(f"{RAW}/tag.txt", **OPTIONS)
tag = tag[tag["tag"].isin(TAGS) & (tag["custom"] == "0")]
tag.to_sql("sec_tag_raw", engine, schema="data", if_exists="replace", index=False)
print("sec_tag_raw :", len(tag))

# 3. Montants : lus par morceaux car le fichier est gros
adsh_10k = set(sub["adsh"])
morceaux = []
for chunk in pd.read_csv(f"{RAW}/num.txt", chunksize=500_000, **OPTIONS):
    chunk = chunk[
        chunk["tag"].isin(TAGS)
        & (chunk["uom"] == "USD")
        & chunk["adsh"].isin(adsh_10k)
        & chunk["version"].str.startswith("us-gaap", na=False)
    ]
    # On garde le total de l'entreprise, pas les ventilations
    for col in ["coreg", "segments"]:
        if col in chunk.columns:
            chunk = chunk[chunk[col].isna()]
    morceaux.append(chunk)
num = pd.concat(morceaux)
num.to_sql("sec_num_raw", engine, schema="data", if_exists="replace",
           index=False, chunksize=10_000)
print("sec_num_raw :", len(num))
```

```
python scripts/load_sec.py
```

Vérification :

```
docker exec gouv_postgres psql -U gouv -d governance -c "SELECT COUNT(*) FROM data.sec_sub_raw;"
docker exec gouv_postgres psql -U gouv -d governance -c "SELECT COUNT(*) FROM data.sec_num_raw;"
```

Notez les nombres de lignes : ce sont des **métadonnées opérationnelles** (volumétrie).

> **CAPTURE D'ÉCRAN REQUISE**
> La sortie du script et les deux `COUNT(*)`.

Commit : `TP1 etape 2 : chargement brut du dataset SEC`

ÉTAPE 3 : Documenter le dataset (métadonnées et dictionnaire)

3.1 La fiche catalogue (niveau dataset)

Créez `docs/dictionnaire/sec_financial_statements.md` et commencez par cette fiche. Complétez les `...`.

Type	Attribut	Valeur
Métier	Nom métier	États financiers annuels des sociétés cotées US
Métier	Description	... (2 phrases : que contient-il, à quoi sert-il)
Métier	Domaine	Finance
Technique	Tables	`data.sec_sub_raw`, `data.sec_num_raw`, `data.sec_tag_raw`
Technique	Format source	TXT séparé par tabulations, encodage UTF-8
Opérationnelle	Source	SEC, Financial Statement Data Sets, trimestre ...
Opérationnelle	Fréquence de mise à jour	Trimestrielle (côté SEC)
Opérationnelle	Volumétrie	... lignes (sub), ... lignes (num)
Opérationnelle	Filtre appliqué	Formulaires 10-K, 8 tags us-gaap, USD, sans coreg ni segments
Gouvernance	Data Owner	Direction Finance
Gouvernance	Data Steward	... (votre nom)
Gouvernance	Classification	Public
Gouvernance	Sensibilité	Low
Gouvernance	PII	Non

3.2 Le dictionnaire de données (niveau colonne)

Documentez **au moins 8 colonnes** : `adsh`, `cik`, `name`, `sic`, `countryba`, `tag`, `ddate`, `uom`, `value`, `qtrs`.

Voici un exemple rempli. Faites les autres sur le même modèle :

Nom physique	Nom métier	Définition	Type	Contraintes	Valeurs admises	Règle de gestion	PII
`cik`	Identifiant SEC de l'entreprise	Numéro unique attribué par la SEC à chaque entité qui dépose des rapports	Texte (10 chiffres)	NOT NULL	10 chiffres, complété par des zéros à gauche	Doit être unique par entreprise dans `master.company`	Non
`qtrs`	Durée de la période	Nombre de trimestres couverts par le montant	Entier	NOT NULL	0 = montant à une date (bilan), 4 = montant sur un an (compte de résultat)	...	Non

> **EXPLICATION : les 4 règles de rédaction du cours**
>
> 1. **La définition** est une phrase complète, sans jargon technique. Pas "champ VARCHAR", mais "numéro unique attribué par la SEC".
> 2. **La règle de gestion** commence par un verbe et se vérifie : "Doit être unique", "Doit être supérieur à 0".
> 3. **Les valeurs admises** sont exhaustives et chaque code a un sens : `0 = bilan, 4 = annuel`.
> 4. **Le propriétaire** est toujours une direction métier, jamais l'IT.

Commit : `TP1 etape 3 : fiche catalogue et dictionnaire SEC`

ÉTAPE 4 : Cataloguer dans OpenMetadata

4.1 Ingestion

1. **Paramètres, Services, Bases de données**, ouvrir `gouv_postgres` (créé au TP0).
2. Onglet **Ingestions** (ou **Agents** selon la version), **Ajouter une ingestion de métadonnées**.
3. Filtre de schémas : inclure `data`, `master`, `ref`.
4. Lancer l'ingestion et attendre le statut **Success**.
5. Dans **Explorer**, chercher `sec_num_raw` : la table et ses colonnes apparaissent.

> **EXPLICATION : ce que l'ingestion a deviné toute seule**
> Les métadonnées **techniques** (noms de colonnes, types) et **opérationnelles** (nombre de lignes, date de mise à jour). Les métadonnées **métier** et **de gouvernance**, aucune machine ne peut les deviner : c'est votre travail de Steward (étapes 4.2 à 4.4).

4.2 Enrichir les 3 tables `sec_*_raw`

Pour chacune :

Champ OpenMetadata	Valeur
Description	Reprendre la description de votre fiche catalogue
Owner	Équipe `Direction Finance`
Domaine	`Finance`
Tags	`Confidentialite.Public`, `Sensibilite.Low`

Sur 3 ou 4 colonnes importantes (`cik`, `tag`, `value`...), ajoutez la description du dictionnaire et le tag `PII.NonSensitive`.

4.3 Glossaire

Dans `Glossaire Projet`, créez ces termes (définition + règle de gestion) :

Terme	Définition	Lier à
Company	Entité juridique cotée qui dépose des rapports à la SEC	table `master.company` (étape 5)
CIK	Identifiant unique d'une entreprise à la SEC	colonne `cik`
Revenue	Chiffre d'affaires : total des ventes de la période	tag XBRL `Revenues`
XBRL Tag	Étiquette standard d'un poste financier	colonne `tag`

> **CAPTURE D'ÉCRAN REQUISE**
> La page de `sec_num_raw` avec owner, domaine, tags et description. La page du glossaire avec vos 4 termes.

ÉTAPE 5 : Construire le MDM Tiers et Structures

5.1 Choisir les entités maîtres avec les 5 tests

Complétez ce tableau dans votre compte rendu (OUI / NON + justification courte) :

Entité	Nature	Volatilité faible	Réutilisation	Partage	Criticité	Master Data ?
Company	...	...	...	...	...	...
Tag XBRL (plan de comptes)	...	...	...	...	...	...
Currency	...	...	...	...	...	...
Un montant de `num.txt`	...	...	...	...	...	...

> **EXPLICATION : le résultat attendu**
> Company, Tag XBRL et Currency réussissent les tests : ce sont des **entités** stables, utilisées partout. Un montant est un **fait** lié à une date : il change à chaque rapport, ce n'est **pas** une Master Data. Il ira dans `data.sec_financial_facts` et **pointera** vers les Master Data.

5.2 Le modèle

```
 ref.country    ref.sic       ref.currency
      \          /                 |
    master.company          master.xbrl_account
     (Golden Record)                |
           \                        |
            +--- data.sec_financial_facts ---+
                (un montant = une ligne)
```

5.3 Le script MDM

Créez `sql/11_finance_mdm.sql` :

```
-- MDM Tiers et Structures (membre Finance)
-- Pre-requis : data.sec_sub_raw, data.sec_num_raw, data.sec_tag_raw charges

-- 1. Reference Data ------------------------------------------------------
CREATE TABLE IF NOT EXISTS ref.currency (
  code  CHAR(3) PRIMARY KEY,
  label VARCHAR(50) NOT NULL);
INSERT INTO ref.currency VALUES
  ('USD','Dollar americain'), ('EUR','Euro'), ('MAD','Dirham marocain')
ON CONFLICT DO NOTHING;

CREATE TABLE IF NOT EXISTS ref.country (
  code  CHAR(2) PRIMARY KEY,
  label VARCHAR(80) NOT NULL);
INSERT INTO ref.country VALUES
  ('US','Etats-Unis'), ('CA','Canada'), ('GB','Royaume-Uni'), ('IE','Irlande'),
  ('CH','Suisse'), ('NL','Pays-Bas'), ('IL','Israel'), ('CN','Chine'),
  ('BM','Bermudes'), ('KY','Iles Caimans'), ('JP','Japon'), ('MA','Maroc')
ON CONFLICT DO NOTHING;

-- Codes secteur SIC de la SEC (libelles a completer, etape 5.5)
CREATE TABLE IF NOT EXISTS ref.sic (
  code  VARCHAR(4) PRIMARY KEY,
  label VARCHAR(150));
INSERT INTO ref.sic (code)
SELECT DISTINCT sic FROM data.sec_sub_raw WHERE sic IS NOT NULL
ON CONFLICT DO NOTHING;

-- 2. Golden Record Company -----------------------------------------------
CREATE TABLE IF NOT EXISTS master.company (
  company_id SERIAL PRIMARY KEY,
  cik        VARCHAR(10) UNIQUE NOT NULL,
  legal_name VARCHAR(200) NOT NULL,
  country    CHAR(2) REFERENCES ref.country(code),
  sic_code   VARCHAR(4) REFERENCES ref.sic(code),
  updated_at TIMESTAMP DEFAULT now());

-- Regle de priorite : pour un meme CIK, le depot le plus recent gagne
-- Standardisation : CIK sur 10 chiffres, nom en majuscules sans espaces autour,
-- pays inconnu du referentiel -> NULL (et signale par le controle qualite)
INSERT INTO master.company (cik, legal_name, country, sic_code)
SELECT DISTINCT ON (cik)
       LPAD(cik, 10, '0'),
       UPPER(TRIM(name)),
       CASE WHEN UPPER(TRIM(countryba)) IN (SELECT code FROM ref.country)
```

```
            THEN UPPER(TRIM(countryba)) END,
       sic
FROM data.sec_sub_raw
ORDER BY cik, filed DESC
ON CONFLICT (cik) DO UPDATE
  SET legal_name = EXCLUDED.legal_name,
      country    = EXCLUDED.country,
      sic_code   = EXCLUDED.sic_code,
      updated_at = now();

-- 3. Plan de comptes XBRL ------------------------------------------------
CREATE TABLE IF NOT EXISTS master.xbrl_account (
  tag      VARCHAR(120) PRIMARY KEY,
  label    VARCHAR(300),
  balance  VARCHAR(6),     -- debit / credit
  category VARCHAR(20) CHECK (category IN
           ('Asset','Liability','Equity','Revenue','Expense','Income')));

-- Si un tag existe dans plusieurs versions de taxonomie, la plus recente gagne
INSERT INTO master.xbrl_account (tag, label, balance, category)
SELECT DISTINCT ON (tag)
       tag, tlabel, crdr,
       CASE
         WHEN tag IN ('Assets','CashAndCashEquivalentsAtCarryingValue') THEN 'Asset'
         WHEN tag = 'Liabilities' THEN 'Liability'
         WHEN tag = 'StockholdersEquity' THEN 'Equity'
         WHEN tag LIKE 'Revenue%' THEN 'Revenue'
         ELSE 'Income'
       END
FROM data.sec_tag_raw
ORDER BY tag, version DESC
ON CONFLICT (tag) DO NOTHING;

-- 4. Faits financiers (donnees metier qui pointent vers le MDM) ------------
CREATE TABLE IF NOT EXISTS data.sec_financial_facts (
  fact_id    BIGSERIAL PRIMARY KEY,
  company_id INT NOT NULL REFERENCES master.company(company_id),
  tag        VARCHAR(120) NOT NULL REFERENCES master.xbrl_account(tag),
  period_end DATE NOT NULL,
  qtrs       SMALLINT,
  value      NUMERIC(24,2),
  currency   CHAR(3) REFERENCES ref.currency(code),
  adsh       VARCHAR(25));

TRUNCATE data.sec_financial_facts;
INSERT INTO data.sec_financial_facts (company_id, tag, period_end, qtrs, value, currency, adsh)
SELECT c.company_id, n.tag, TO_DATE(n.ddate, 'YYYYMMDD'),
       n.qtrs::SMALLINT, n.value::NUMERIC, n.uom, n.adsh
FROM data.sec_num_raw n
JOIN data.sec_sub_raw s ON s.adsh = n.adsh
JOIN master.company c   ON c.cik = LPAD(s.cik, 10, '0')
JOIN master.xbrl_account x ON x.tag = n.tag
WHERE n.value IS NOT NULL;
```

Exécutez-le :

```
docker exec gouv_postgres psql -U gouv -d governance -v ON_ERROR_STOP=1 -f /sql/11_finance_mdm.sql
```

À la deuxième exécution, des messages `NOTICE: ... already exists, skipping` apparaissent : c'est normal, le script est prévu pour être relancé sans danger.

> **EXPLICATION : les 5 étapes du Golden Record, dans le script**
>
> 1. **Charger** les entreprises de la source (une ligne par dépôt dans `sec_sub_raw`).
> 2. **Standardiser** : CIK sur 10 chiffres (`LPAD`), nom en majuscules (`UPPER(TRIM())`), pays au code ISO.
> 3. **Dédoublonner** : `DISTINCT ON (cik)` garde une seule ligne par entreprise.
> 4. **Appliquer la règle de priorité** : `ORDER BY cik, filed DESC`, donc le dépôt le plus récent gagne.
> 5. **Publier** : `master.company` est la fiche unique, et les faits pointent vers elle par `company_id`.

5.4 Le lien avec le membre Investment

Le membre Investment va relier chaque titre boursier (ticker) à **votre** `master.company`, en passant par le CIK. C'est le seul lien naturel entre deux datasets du projet, et c'est **le cœur du MDM** : SEC et S&P 500 parlent des mêmes entreprises, mais une seule fiche fait autorité.

Votre engagement envers lui : la structure de `master.company` (`company_id`, `cik` sur 10 chiffres, `legal_name`, `country`, `sic_code`) ne change plus sans le prévenir. Écrivez-la dans le README, section "Contrats entre membres".

5.5 Contrôles qualité du référentiel

Créez `sql/12_finance_quality.sql` et exécutez-le de la même façon :

```
-- Q1. Unicite : un CIK n'apparait jamais deux fois (attendu : 0 ligne)
SELECT cik, COUNT(*) FROM master.company GROUP BY cik HAVING COUNT(*) > 1;

-- Q2. Pourquoi une regle de priorite ? Entreprises avec plusieurs noms dans la source
SELECT cik, COUNT(DISTINCT name) AS nb_noms
FROM data.sec_sub_raw GROUP BY cik HAVING COUNT(DISTINCT name) > 1;

-- Q3. Completude des attributs du Golden Record (seuil : 95 %)
SELECT COUNT(*) AS nb_companies,
       ROUND(100.0 * COUNT(country)  / COUNT(*), 2) AS pct_country,
       ROUND(100.0 * COUNT(sic_code) / COUNT(*), 2) AS pct_sic
FROM master.company;

-- Q4. Validite : codes pays de la source absents du referentiel
SELECT DISTINCT countryba FROM data.sec_sub_raw
WHERE UPPER(TRIM(countryba)) NOT IN (SELECT code FROM ref.country);

-- Q5. Doublons de faits (meme entreprise, meme poste, meme periode)
SELECT company_id, tag, period_end, qtrs, COUNT(*)
FROM data.sec_financial_facts
GROUP BY company_id, tag, period_end, qtrs HAVING COUNT(*) > 1
LIMIT 10;

-- Q6. Libelles SIC manquants (les 10 codes les plus utilises)
SELECT c.sic_code, COUNT(*) FROM master.company c
JOIN ref.sic s ON s.code = c.sic_code
WHERE s.label IS NULL GROUP BY c.sic_code ORDER BY 2 DESC LIMIT 10;
```

Ce que vous devez faire avec les résultats (c'est le travail du Steward) :

- **Q3 sous 95 %** pour `country` ? Regardez Q4, ajoutez les codes pays manquants dans `ref.country`, relancez `11_finance_mdm.sql`. Notez le avant / après.
- **Q5 renvoie des lignes ?** Expliquez pourquoi (une entreprise a pu déposer un rapport corrigé) et proposez une règle (ex. garder le dépôt le plus récent).
- **Q6** : complétez les libellés des 10 codes SIC les plus fréquents avec la liste officielle de la SEC (https://www.sec.gov/search-filings/standard-industrial-classification-sic-code-list), par des `UPDATE ref.sic SET label = '...' WHERE code = '...';` ajoutés à la fin de `11_finance_mdm.sql`.

> **CAPTURE D'ÉCRAN REQUISE**
> Les résultats de Q1 à Q4, et `SELECT * FROM master.company LIMIT 10;`.

5.6 Mettre à jour le catalogue

1. Relancez l'ingestion OpenMetadata : `master.company`, `master.xbrl_account`, `ref.*` et `data.sec_financial_facts` apparaissent.
2. Documentez `master.company` : description "*Golden Record des entreprises : une ligne par CIK, règle de priorité : dépôt le plus récent*", owner `Direction Finance`, domaine `Finance`, tag `Confidentialite.Internal` (un référentiel interne), `Sensibilite.Low`.
3. Liez le terme de glossaire **Company** à `master.company`.
4. (Optionnel) Onglet **Lignage** de `master.company` : ajoutez à la main `data.sec_sub_raw` → `master.company` → `data.sec_financial_facts`.

> **REMARQUE : pourquoi `master.company` est Internal alors que la SEC est Public ?**
> La donnée source est publique, mais le **référentiel** est le résultat de **vos** règles (nettoyage, priorité). C'est un actif interne de l'entreprise. La classification s'applique à chaque table, pas seulement au dataset d'origine.

Commit : `TP1 etape 5 : MDM Tiers et Structures et controles qualite` Puis ouvrez une **Pull Request** vers `main`, relue par le membre Investment.

Questions de compréhension (à préparer pour l'oral)

1. Pourquoi un montant de `num.txt` n'est-il pas une Master Data ?
2. Quelle est votre règle de priorité ? Donnez un cas réel tiré de Q2 où elle s'applique.
3. Pourquoi `cik` est-il stocké sur 10 chiffres avec des zéros à gauche ?
4. Que se passerait-il pour l'Investment Agent si deux fiches `master.company` existaient pour Apple ?
5. Quelles métadonnées OpenMetadata a-t-il trouvées seul, et lesquelles avez-vous ajoutées ?
6. Pourquoi le filtre "10-K uniquement" doit-il apparaître dans la fiche catalogue ?

Livrables

- [ ] `scripts/explore_sec.py`, `scripts/load_sec.py`
- [ ] `docs/dictionnaire/sec_financial_statements.md` (fiche + 8 colonnes minimum)
- [ ] `sql/11_finance_mdm.sql`, `sql/12_finance_quality.sql`
- [ ] Tableau des 5 tests rempli
- [ ] Captures d'écran des étapes 1, 2, 4 et 5

- [ ] Pull Request relue par le membre Investment

Suite (partie 2) : tables de gouvernance, Governance Engine, Finance Agent.